

Multi-level feedback queue scheduling

A multi-level feedback queue (MLFQ) uses several ready queues, each with its own priority and time quantum, and moves a process between them based on how it actually behaves — not on any burst time given up front.

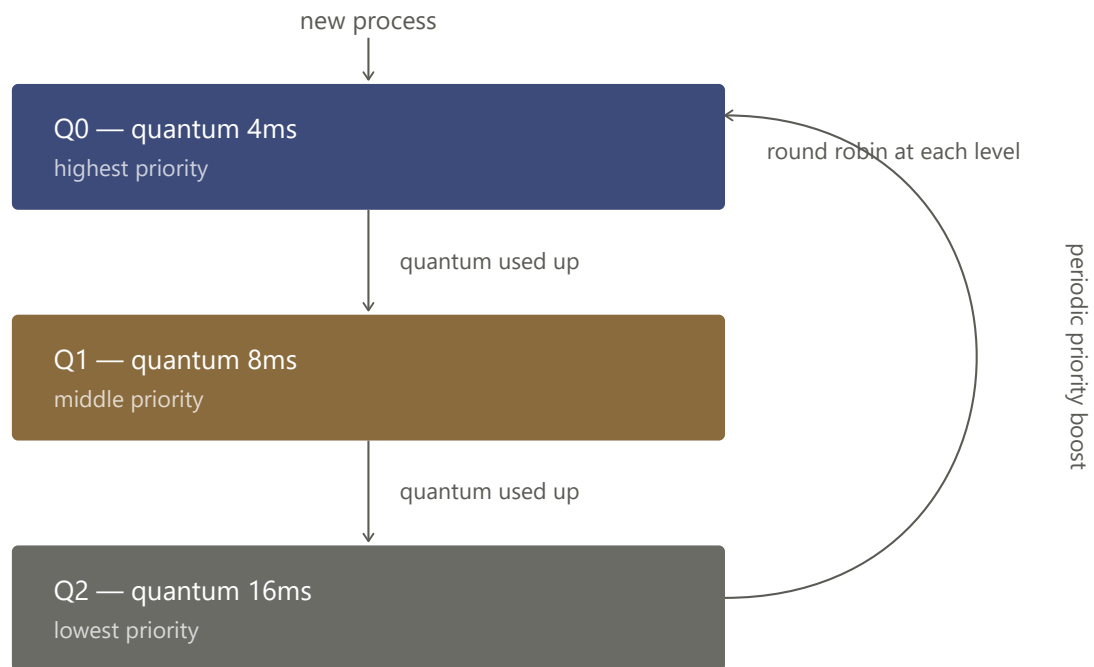
Goals

- Give short, interactive jobs low turnaround time, the way SJF or STCF would — but without knowing burst lengths ahead of time.
- Keep the system feeling responsive, the way round robin does, so no job waits unreasonably long for its first slice of CPU.
- Prevent long-running jobs from being starved of CPU time entirely.

Rules

1. A higher-priority queue always runs before a lower-priority one.
2. Jobs at the same priority level are scheduled round robin, using that level's quantum.
3. A new job enters at the topmost (highest-priority) queue.
4. If a job uses up its entire quantum at a level, it is demoted one level; if it gives up the CPU voluntarily before the quantum expires, it stays where it is.
5. After a fixed period, every job is boosted back to the top queue, so a long job can't be starved indefinitely by a steady stream of new short ones.

Structure



A job that gives up the CPU on its own — waiting on I/O, say — keeps its current level instead of being demoted, so genuinely interactive jobs stay near the top even after running for a while.

Advantages

- Approximates shortest-job-first behaviour for short jobs without ever needing to know burst lengths in advance — a job's history in the queues stands in for that knowledge.
- Stays responsive: because new and short jobs sit in a high queue with a short quantum, they get CPU time quickly.
- The priority boost stops CPU-bound jobs from starving once enough interactive jobs are around.
- Accounting quantum usage per level, rather than per individual time slice, keeps a job from gaming the scheduler by yielding just before its quantum expires.

Worked example

Three processes, one long and two short, arriving a few milliseconds apart, using the queues above (Q0 quantum 4ms, Q1 quantum 8ms, Q2 quantum 16ms):

Process	Arrival	Burst
P1	0	24
P2	4	6
P3	8	2

P1	P2	P3			
----	----	----	--	--	--

P1	P2	P3	P1	P2	P1
Q0	Q0	Q0	Q1	Q1	Q2

0	4	8	12	16	20	24	28	32
---	---	---	----	----	----	----	----	----

P1 starts at Q0 and burns through its 4ms quantum, so it's demoted to Q1. P2 arrives and does the same. P3 arrives with only 2ms of work left, finishes inside its first Q0 quantum, and never gets demoted at all. Once Q0 empties, P1 and P2 run out their Q1 quantum — P1 gets demoted again to Q2, while P2 finishes with only 2ms left to give. P1 spends the rest of its life alone in Q2.

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	24	32	32	8
P2	4	6	20	16	10
P3	8	2	10	2	0

Average turnaround Average waiting
16.7 **6.0**

Analysis against other algorithms

Algorithm	Needs burst length upfront	Responsiveness	Starvation risk
FCFS	No	Poor	None, but everyone queues behind whoever arrived first
SJF	Yes	Poor	Long jobs can starve behind a stream of short ones
Round robin	No	Good	None

Algorithm	Needs burst length upfront	Responsiveness	Starvation risk
Static priority	No	Depends on assignment	Low-priority jobs can starve
MLFQ	No	Good	Mitigated by the periodic boost

Running the same three processes through FCFS, non-preemptive SJF, and plain round robin (quantum 4) makes the trade-offs concrete:

Algorithm	Average turnaround	Average waiting
FCFS	24.7	14.0
SJF (non-preemptive)	23.3	12.7
Round robin (q=4)	16.7	6.0
MLFQ	16.7	6.0

MLFQ ties plain round robin on the average here, but the two don't treat individual jobs the same way. Looking at waiting time process by process shows why:

Algorithm	P1 wait	P2 wait	P3 wait
FCFS	0	20	22
SJF (non-preemptive)	0	22	16
Round robin (q=4)	8	6	4
MLFQ	8	10	0

Non-preemptive SJF actually does worse than either RR or MLFQ for P3 here: P1 arrives first and, having no preemption to interrupt it, gets to run for its full 24ms before the scheduler ever gets to compare burst lengths — the classic convoy effect. MLFQ avoids that because it's preemptive from the start: P3's arrival lets it grab a Q0 slot the moment it shows up, so it waits 0ms despite the scheduler never being told it was the shortest job. Against round robin, MLFQ shifts wait time away from the shortest job (P3, 0 vs 4) and onto the medium one (P2, 10 vs 6), while landing on the same average — a small piece of evidence that MLFQ's queue structure does approximate SJF-like ordering, not just plain fairness.

MLFQ's real advantage over both SJF and static priority is that it never has to assume anything about a job upfront — it just watches how long a job keeps running and reacts to that, then uses the periodic boost as a safety valve so that reactivity never turns into starvation.